

# Vertical Order Traversal of a Binary Tree

---

Difficulty: Hard

Patterns: Tree, Binary Tree, Depth-First Search, Breadth-First Search, Hash Table, Sorting

LeetCode: <https://leetcode.com/problems/vertical-order-traversal-of-a-binary-tree/>

## 1. Problem Summary

---

Given the root of a binary tree, return its vertical order traversal.

Imagine the root at coordinate  $(row = 0, col = 0)$ .

- A left child moves to  $(row + 1, col - 1)$ .
- A right child moves to  $(row + 1, col + 1)$ .

The answer should list columns from leftmost to rightmost. Inside one column, nodes should appear from top to bottom. If multiple nodes share the exact same row and column, sort those node values in ascending order.

Example:

- Input: `root = [1,2,3,4,5,6,7]`
- Output: `[[4], [2], [1,5,6], [3], [7]]`

The important tie is at  $(row = 2, col = 0)$ : nodes 5 and 6 are in the same position, so they appear as 5, 6.

## 2. Constraints and Observations

---

- Constraint: The number of nodes is in the range  $[1, 1000]$ . What it implies: An  $O(n \log n)$  sorting solution is comfortably accepted.
- Constraint:  $0 \leq \text{Node.val} \leq 1000$ . What it implies: Node values are small, but the simplest interview-ready solution should still use normal sorting.
- Observation: Every node can be described by a triple  $(column, row, value)$ .

- Observation: The required output order is exactly the sorted order of these triples: first by column, then by row, then by value.
- Observation: This is not the same as simple vertical order traversal. BFS order alone does not handle the same-row same-column value tie.

### 3. Pattern Recognition

---

This is a tree traversal plus sorting problem.

The traversal itself can be DFS or BFS because both can visit every node and assign coordinates. The ordering requirement is handled after traversal by sorting coordinates.

The key pattern is:

1. Convert structure into sortable records.
2. Sort records by the problem's priority rules.
3. Group adjacent records with the same column.

### 4. Naive Approach

---

A naive idea is to do a normal level-order traversal and append each value into a map keyed by column.

That works for LeetCode 314 style vertical order traversal, but it fails here because of the tie rule.

Consider:



Both 6 and 5 are at  $(row = 2, col = 0)$ . If BFS sees 6 before 5, appending by traversal order gives  $[1, 6, 5]$  in column 0, which is wrong. The correct order is  $[1, 5, 6]$ .

The bottleneck is not visiting nodes. The bottleneck is enforcing the exact ordering rule.

## 5. Key Intuition

The problem statement gives us the sorting key directly.

For every node, record:

```
(column, row, value)
```

Then sort all records. Python tuple sorting already compares in this order:

1. Smaller column first.
2. If columns tie, smaller row first.
3. If row also ties, smaller value first.

After sorting, all nodes from the same column are next to each other, already in the correct vertical order.

## 6. Optimal Solutions Ranked

| Rank | Variant                                    | Idea                                                                           | Time          | Space  | Tradeoff                                                                        |
|------|--------------------------------------------|--------------------------------------------------------------------------------|---------------|--------|---------------------------------------------------------------------------------|
| 1    | BFS map by column, then sort inside groups | Track <code>(row, value)</code> per column and sort each column's list.        | $O(n \log n)$ | $O(n)$ | Good, but requires sorting columns and then sorting each group.                 |
| 2    | DFS collect triples, global sort           | Store <code>(col, row, val)</code> for every node, sort once, group by column. | $O(n \log n)$ | $O(n)$ | Cleanest and least error-prone; final code uses this.                           |
| 3    | Priority queues per column/row             | Push values into heaps keyed by <code>(col, row)</code> , then emit in order.  | $O(n \log n)$ | $O(n)$ | Useful in streaming-style designs, but more moving parts for no asymptotic win. |

Under normal comparison sorting,  $O(n \log n)$  is the practical optimal bound because the answer may require sorting many values that share the same row and column. The output itself stores all  $n$  values, so  $O(n)$  space is unavoidable.

Although node values are bounded from 0 to 1000, using counting buckets for every coordinate adds complexity and is not the standard interview solution. The DFS plus global sort approach is the best balance of correctness, clarity, and performance.

## 7. Most Optimal Approach

---

Use DFS to assign coordinates to every node:

- Root starts at  $(row = 0, col = 0)$ .
- Left child becomes  $(row + 1, col - 1)$ .
- Right child becomes  $(row + 1, col + 1)$ .

Store each node as  $(col, row, value)$ . Sort the full list, then scan from left to right and start a new output column whenever the column value changes.

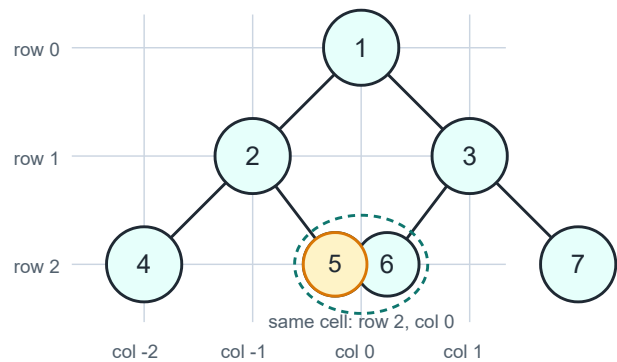
## 8. Algorithm

---

1. Create an empty list `nodes`.
2. Define `dfs(node, row, col)`:
  1. If `node` is `None`, return.
  2. Append  $(col, row, node.val)$  to `nodes`.
  3. Recurse left with  $(row + 1, col - 1)$ .
  4. Recurse right with  $(row + 1, col + 1)$ .
3. Run `dfs(root, 0, 0)`.
4. Sort `nodes`.
5. Initialize an empty answer list.
6. Scan through sorted triples:
  1. If this is a new column, append a new empty list to the answer.
  2. Add the current value to the latest column list.
7. Return the answer.

## 9. Visual Explanation

Assign each node a coordinate, then sort by column, row, value



### Collected triples

Store each node as (column, row, value).

#### Before sorting

(0,0,1), (-1,1,2), (1,1,3)  
(-2,2,4), (0,2,6), (0,2,5), (2,2,7)

#### After sorting by col, row, value

(-2,2,4), (-1,1,2), (0,0,1)  
(0,2,5), (0,2,6), (1,1,3), (2,2,7)

**Result columns:** [[4], [2], [1, 5, 6], [3], [7]]

The tie at (row 2, col 0) is why traversal order alone is not enough; values 5 and 6 must be sorted.

Figure 1: Each node becomes a sortable coordinate record. Sorting by `(column, row, value)` automatically handles the same-position tie.

## 10. Dry Run

Use:

```
root = [1,2,3,4,6,5,7]
```

Tree:



Collected triples:

| Node | Row | Column | Stored triple |
|------|-----|--------|---------------|
| 1    | 0   | 0      | (0, 0, 1)     |
| 2    | 1   | -1     | (-1, 1, 2)    |
| 3    | 1   | 1      | (1, 1, 3)     |
| 4    | 2   | -2     | (-2, 2, 4)    |
| 6    | 2   | 0      | (0, 2, 6)     |
| 5    | 2   | 0      | (0, 2, 5)     |
| 7    | 2   | 2      | (2, 2, 7)     |

After sorting:

```
[(-2, 2, 4), (-1, 1, 2), (0, 0, 1), (0, 2, 5), (0, 2, 6), (1, 1, 3), (2, 2, 7)]
```

Group by column:

```
[[4], [2], [1, 5, 6], [3], [7]]
```

## 11. Correctness Argument

We prove the algorithm returns the required vertical traversal.

**Coordinate correctness:** The DFS starts the root at  $(0, 0)$ . For every node at  $(row, col)$ , the algorithm visits its left child with  $(row + 1, col - 1)$  and its right child with  $(row + 1, col + 1)$ . This exactly matches the coordinate definition in the problem, so every stored triple has the correct column and row.

**Ordering correctness:** The required order compares nodes first by column, then by row, then by value for exact ties. The algorithm stores triples as  $(column, row, value)$  and sorts them. Tuple sorting therefore places every node in the exact required order.

**Grouping correctness:** After sorting, all nodes with the same column appear contiguously because column is the first sort key. The scan starts a new list only when the column changes, so each output

list contains exactly one column, and columns appear from leftmost to rightmost.

Therefore, the returned answer is exactly the vertical order traversal required by the problem.

## 12. Complexity Analysis

---

- Time:  $O(n \log n)$ , where  $n$  is the number of nodes. DFS visits every node once, and sorting the  $n$  triples dominates the runtime.
- Space:  $O(n)$ . The list of triples stores one record per node, and the returned answer stores all node values. The recursive call stack can be  $O(h)$ , where  $h$  is the tree height, which is  $O(n)$  in the worst case.

## 13. Edge Cases

---

- Single node: returns `[[root.val]]`.
- Perfect binary tree: many columns and same-position ties can occur; sorting handles them.
- Same row and same column with reversed values: values are sorted ascending, independent of traversal order.
- Skewed tree: each node may create a new column; grouping still works.
- Duplicate values: duplicates remain in the column output because we store every node separately.

## 14. Final Clean Code

---

```
from typing import List, Optional

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def verticalTraversal(self, root: Optional[TreeNode]) -> List[List[int]]:
        nodes = []

        def dfs(node: Optional[TreeNode], row: int, col: int) -> None:
            if node is None:
                return

            nodes.append((col, row, node.val))
            dfs(node.left, row + 1, col - 1)
            dfs(node.right, row + 1, col + 1)

        dfs(root, 0, 0)
        nodes.sort()

        result = []
        previous_col = None

        for col, row, value in nodes:
            if col != previous_col:
                result.append([])
                previous_col = col
            result[-1].append(value)

        return result
```

## 15. Key Takeaways

---

- Main pattern: Tree traversal plus sorting.
- Main trick: Convert every node into (column, row, value).



- Interview explanation: "I assign each node a grid coordinate, sort by the exact priority the problem asks for, then group adjacent nodes with the same column."